LAB W11 · ONE LAB, ELEVEN STEPS, EACH ONE BUILDS ON THE LAST

Happy Tails: a shelter API with a front door

The Happy Tails animal shelter needs an API. Anyone should be able to browse the animals. Only shelter staff, holding a token, should be able to add or change one. And only the manager, with a username and password, should see the numbers. Tonight you build all of that in one Next.js app, one step at a time, testing each step in Postman before you move on.

This lab is meant to stretch you. Every step tells you what to build and how to check it. If you get stuck, every step also links to the full code for that step in the Code reference at the back. Try it yourself first, then look.

Contents

0. Run it and meet the data
1. Public: the list of pets
2. Public: one pet
3. POST: add a pet
4. PUT: replace a pet
5. Private: a token in `.env.local`
6. `proxy.ts`: one file before every route
7. Move the token check into the proxy
8. Let the public routes back in
9. Logging that helps
10. Errors that help
11. The manager's door: Basic auth

What "done" looks like · Going further · Things that will bite you · Code reference

What you'll have at the end

ADDRESS	VERB	WHO CAN CALL IT	WHAT IT DOES
`/api/health`	GET	Anyone	Says the server is up (already done)
`/api/pets`	GET	Anyone	Every pet
`/api/pets/2`	GET	Anyone	One pet
`/api/pets`	POST	Staff (Bearer token)	Add a pet
`/api/pets/2`	PUT	Staff (Bearer token)	Replace a pet
`/api/admin/summary`	GET	Manager (username and password)	How many pets, adopted, waiting

USING CLAUDE IN THIS LAB

You have Claude chat open, so use it, but use it to **understand**, not to paste. Start every new chat by pasting this, so it knows exactly what you're working with:

> *"I'm a beginner in a full-stack class. I'm building an API-only Next.js 16 app (App Router, TypeScript, no pages). Routes are files at app/api/.../route.ts that export functions named GET, POST and PUT. In Next 16, middleware.ts was renamed to proxy.ts: it sits in the project's top folder and exports a function called proxy. My data is an in-memory array in types/pet.ts, no database and no extra libraries. When I ask for help, explain the idea and give me a hint or a small piece first, not a whole finished file, unless I say I'm completely stuck."*

When something breaks, give it three things: **the file**, **what the terminal printed**, and **what you expected vs. what Postman showed** (the status and the body). "It doesn't work" gets you a guess. Those three get you an answer.

If Claude ever suggests `middleware.ts`, `pages/api`, `req, res`, `NextApiRequest` or Express, it's describing an older Next or a different tool. Remind it: Next 16, App Router, `proxy.ts`.

0 Run it and meet the data

```
cd mtc-ai-demos/week11/lab1
npm install
npm run dev
```

Keep that terminal where you can see it for the whole lab. When something goes wrong, Postman often just shows a status, and the reason is printed in the terminal.

Open the **Postman desktop app** (the web version can't reach `localhost`). Make a collection called **Happy Tails** and save every request you make into it (`Ctrl` + `S`). Later steps ask you to re-run earlier requests to check you didn't break them, and a saved collection makes that a few clicks.

Now read `types/pet.ts`. It's like the coffee menu from lecture: a type that says what a pet looks like, and a `const` array of pets that **is** the server's data. Nothing is stored anywhere else. When a route pushes onto that array or changes a pet in it, the very next request sees the change. That's what makes this a live model you can watch change from Postman.

```
export type Pet = {
  id: number;
  name: string;
  species: string;
  age: number;
  adopted: boolean;
};

export const allowedSpecies = ["dog", "cat", "rabbit"];

export const pets: Pet[] = [
  { id: 1, name: "Biscuit", species: "dog", age: 3, adopted: false },
  ...
];
```

One route is already done for you, `app/api/health/route.ts`. Read it. That's the whole shape of a route: the folder is the address, the function's name is the verb.

TEST IN POSTMAN

SEND	AUTH / BODY	EXPECT	BODY
`GET http://localhost:3000/api/health`	none	**200**	`{"status": "ok"}`

Also open http://localhost:3000 in a browser. You get a 404 page. That's fine: this app has no pages at all, only an API.

ASK YOUR CLAUDE

> *"Here's my types/pet.ts: [paste it]. In plain words, what does* `export const pets: Pet[]` *mean? If a route on the server pushes a new pet onto this array, why does the next request see it, and why would it disappear when I restart the server?"*

1 Public: the list of pets

Make `app/api/pets/route.ts`. Because of its folder, it answers at `/api/pets`. Export a function named `GET` that returns the whole `pets` array with `Response.json(...)`.

Import `pets` from `types/pet.ts`. Count the folders: from `app/api/pets/` you go up three (`../../../`) to reach the top folder.

```
import { pets } from "../../../types/pet";

export async function GET() {
  // return the array as JSON
}
```

TEST IN POSTMAN

SEND	AUTH / BODY	EXPECT	BODY
`GET /api/pets`	none	**200**	All 6 pets, Biscuit through Waffles

Got a **405**? The function's name isn't exactly `GET` (all capitals). Next routes on that name.

Stuck? Code C1: the list route

ASK YOUR CLAUDE

> *"In a Next 16 route.ts file, why does the name of the exported function matter? What would happen if I named it* `handler` *instead of* `GET`*, and what status code would I see?"*

2 Public: one pet

Make `app/api/pets/[id]/route.ts`. The square brackets mean "any value goes here," so this one file answers `/api/pets/1`, `/api/pets/2`, and so on. Next hands your function that value as its second input:

```
export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  // id is text: "2", not 2
}
```

Find the pet whose `id` matches. Watch out: `id` from the address is text, and the ids in the array are numbers, so `"2" === 2` is `false`. Turn it into a number with `Number(id)` first.

If there's no such pet, `find` gives you `undefined`. Don't let that slide through. Answer **404** yourself, with an error body in the same shape every time:

```
return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
```

This file is one folder deeper, so the import goes up **four** folders.

TEST IN POSTMAN

SEND	AUTH / BODY	EXPECT	BODY

`GET /api/pets/2`	none	**200**	`{"id": 2, "name": "Miso", "species": "cat", ...}`
`GET /api/pets/99`	none	**404**	`{"error": "No pet with id 99"}`
`GET /api/pets/abc`	none	**404**	`{"error": "No pet with id abc"}`

A **500** for `/api/pets/99` means you returned `Response.json(undefined)`.
Add the "not found" check.

Stuck? Code C2: the one-pet route

ASK YOUR CLAUDE

> *"In Next 16 route handlers, why is `params` a Promise that I have to `await`? And why is `id` a string even when the URL is /api/pets/2?"*

> *"Why is a missing pet a 404 and not a 500? Whose fault is each one, the client's or the server's?"*

3 POST: add a pet

Go back to `app/api/pets/route.ts` and add a second function, `POST`, next to `GET`. Same address, different verb, different function.

Read the body with `await request.json()`. It **throws** if the body isn't JSON, so wrap it in `try` / `catch` and answer 400:

```
let body;
try {
  body = await request.json();
} catch {
  return Response.json({ error: "The body must be JSON" }, { status: 400 });
}
```

Then **check every field before you save anything**. Each failed check is a 400 with its own message:

- `name` must be a string and not empty. Use `typeof body.name !== "string"`.
- `species` must be one of `allowedSpecies`. Import it too, and use `allowedSpecies.includes(body.species)`.
- `age` must be a number, 0 or more.

Only when all three pass do you build the new pet: `id` is `pets.length + 1`, `adopted` starts as `false`. Push it onto `pets` and return it with status **201** ("created"). `Response.json` always says 200 unless you tell it otherwise.

TEST IN POSTMAN

For each POST: **Body** tab, then **raw**, then pick **JSON** from the dropdown on the right.

SEND	AUTH / BODY	EXPECT	BODY
`POST /api/pets`	`{"name": "Nugget", "species": "dog", "age": 2}`	**201**	`{"id": 7, "name": "Nugget", ..., "adopted": false}`
`GET /api/pets`	none	**200**	7 pets now. Nugget is on the end.
`GET /api/pets/7`	none	**200**	Nugget. The other file sees the same array.
`POST /api/pets`	`{}`	**400**	`{"error": "name is required"}`
`POST /api/pets`	`{"name": "Rex", "species": "dragon", "age": 2}`	**400**	`{"error": "species must be dog, cat or rabbit"}`
`POST /api/pets`	`{"name": "Rex", "species": "dog", "age": "two"}`	**400**	`{"error": "age must be a number, 0 or more"}`
`POST /api/pets`	Switch JSON to **Text** and send the word `banana`	**400**	`{"error": "The body must be JSON"}`

Now stop the server (`Ctrl` + `C`), start it again, and `GET /api/pets` . Nugget is gone. The array lives in memory, so a restart puts it back to what the file says. (Next week it moves into a database.)

HOLD THAT THOUGHT

Right now, **anyone** who can reach this server can add a pet. Nothing asked who you were. You'll fix that in Step 5.

Stuck? Code C3: the list route with POST

ASK YOUR CLAUDE

> *"I'm checking a POST body by hand with typeof. What does* `typeof` *return for "two", for 2, and for undefined? Why does* `"age": "4"` *fail a number check but* `"age": 4` *pass?"*

> *"How does* `Array.includes` *work? Show me with* `["dog", "cat", "rabbit"]` *and the values "cat", "Cat" and undefined."*

4 PUT: replace a pet

Add a `PUT` function to `app/api/pets/[id]/route.ts`, next to its `GET`. It takes the same two inputs as `GET` (the `request` and the `params`).

> **PUT VS. PATCH**
>
> **PUT** means "here is the whole new pet, replace the old one." The client must send **every** field. **PATCH** (what lecture did) means "change only the fields I sent." Tonight is PUT, so a body missing any field is a 400.

In order, the function should:

1. Find the pet, exactly like `GET` does. Not found: 404.
2. Read the body in a `try` / `catch`, exactly like `POST`. Not JSON: 400.
3. Run the same three checks as `POST`, plus a fourth: `adopted` must be `true` or `false` (`typeof body.adopted !== "boolean"`).
4. Change the pet you found (`pet.name = body.name`, and the same for the other three) and return it with **200**. Leave its `id` alone.

Copying the checks from `POST` is fine. Recognizing a pattern you already wrote is the point.

TEST IN POSTMAN

SEND	AUTH / BODY	EXPECT	BODY
`PUT /api/pets/3`	`{"name": "Pepper", "species": "rabbit", "age": 1, "adopted": true}`	**200**	Pepper, now `"adopted":true`
`GET /api/pets/3`	none	**200**	Still adopted. The change stuck.
`PUT /api/pets/3`	`{"adopted": true}`	**400**	`{"error": "name is required"}`. That's PUT doing its job.
`PUT /api/pets/3`	the full body, but `"adopted":"yes"`	**400**	`{"error": "adopted must be true or false"}`
`PUT /api/pets/99`	the full body	**404**	`{"error": "No pet with id 99"}`

CHECKPOINT

You now have GET, POST and PUT all working on one live list. Every route works, and every route is wide open. The rest of the lab is about deciding who gets in.

Stuck? Code C4: the one-pet route with PUT

ASK YOUR CLAUDE

> *"Explain PUT vs. PATCH using a pet record as the example. If a client PUTs only* `{"adopted": true}`*, what should a strict PUT do, and why is that safer than guessing?"*

5 Private: a token in `.env.local`

Staff will prove who they are with a **token**: a secret string sent with every request. The token is a secret, so it never goes in your code. It goes in an environment file.

5a. Make sure Git will ignore it

Open `.gitignore` at the top of the repo and find the `.env*` line. That's what keeps env files out of Git. Every project you make from now on needs that line **before** you create the file.

5b. Make `.env.local`

Make a file called `.env.local` in `week11/lab1/`, right next to `package.json` (not inside `app/`):

```
API_TOKEN=shelter-staff-2026
```

No quotes, no spaces around the `=`. Next picks it up without a restart. The terminal prints `Reload env: .env.local`. Your code reads it as `process.env.API_TOKEN`. Never give it a `NEXT_PUBLIC_` prefix. That prefix means "paste this into the browser's code," which is exactly where a secret must not go.

5c. How the token is sent

Lecture used a made-up header called `x-auth`. The standard place for a token is the `Authorization` header, and the value is the word `Bearer`, a space, then the token. "Bearer" means "whoever bears (carries) this token gets in."

```
Authorization: Bearer shelter-staff-2026
```

5d. Check it in POST

At the very top of your `POST` function, before reading the body, compare the header to what it should be. If it doesn't match, answer **401** ("I don't know who you are") and stop:

```
if (request.headers.get("authorization") !== `Bearer ${process.env.API_TOKEN}`) {
  return Response.json({ error: "Missing or wrong token" }, { status: 401 });
}
```

TEST IN POSTMAN

To send the token: **Authorization** tab, **Auth Type** = **Bearer Token**, paste `shelter-staff-2026` into **Token**. Then look at the **Headers** tab and show the hidden headers. There's the `Authorization: Bearer ...` line Postman built for you.

SEND	AUTH / BODY	EXPECT	BODY
`POST /api/pets`	No Auth, Nugget body	**401**	`{"error": "Missing or wrong token"}`
`POST /api/pets`	Bearer `wrong`, Nugget body	**401**	`{"error": "Missing or wrong token"}`
`POST /api/pets`	Bearer `shelter-staff-2026`, Nugget body	**201**	Nugget, id 7
`PUT /api/pets/3`	**No Auth**, the full body	**200**	It worked, with no token at all. Why?

THE PROBLEM WITH THIS

You locked one door. `PUT` is still open. You could paste the same check into `PUT`, and into every route you write next month, and hope you never forget one. Forgetting one is how real apps get broken into. Next step: put the check in **one place** that every request has to pass through.

Stuck? Code C5: the token check at the top of POST

ASK YOUR CLAUDE

> *"What is the Authorization header, and what does the word Bearer mean in* `Authorization: Bearer abc123`*? Why is a header better than putting the token in the URL, like* `?token=abc123`*?"*

> *"In Next.js, what's the difference between* `process.env.API_TOKEN` *and* `process.env.NEXT_PUBLIC_API_TOKEN`*? Where can each one be seen?"*

6 `proxy.ts`: one file before every route

`proxy.ts` is a single file whose function runs **before every request** reaches any route. Start it the way lecture did, as a logger only, so you can prove it's running before you trust it with anything.

Make `proxy.ts` in the **top folder** of the project, next to `package.json`, **not** inside `app/`:

```
week11/lab1/
  app/
  types/
  .env.local
  package.json
  proxy.ts      <-- here
```

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  console.log("proxy saw:", request.method, request.nextUrl.pathname);
  return NextResponse.next();
}
```

`NextResponse.next()` means "carry on to the route." Right now the proxy lets everything through. It only watches.

TEST IN POSTMAN, THEN READ THE TERMINAL

SEND	AUTH / BODY	EXPECT	TERMINAL SHOWS
`GET /api/pets`	none	**200**	`proxy saw: GET /api/pets`
`DELETE /api/pets/1`	none	**405**	`proxy saw: DELETE /api/pets/1`, even though no route answers DELETE
`GET /api/nope`	none	**404**	`proxy saw: GET /api/nope`, even for an address that doesn't exist

The proxy sees **everything**, before Next even knows whether a route exists. That's why it's the right place for checks every route needs.

NO `PROXY SAW:` LINE?

The file is in the wrong folder. Anywhere other than the top folder, it's an ordinary file nothing calls, and Next doesn't warn you. Also, the line that looks like `GET /api/pets 200 in 12ms` is Next's own log, not yours. Yours starts with `proxy saw:`.

Stuck? Code C6: the logging proxy

ASK YOUR CLAUDE

> *"My Next 16 proxy.ts doesn't seem to run: no console.log appears and there's no error. Here's my folder layout: [paste it]. Where exactly does Next 16 look for proxy.ts, and what is it called in older versions?"*

7 Move the token check into the proxy

Now give the proxy a job. Copy your token check from `POST` into the proxy, between the log line and `return NextResponse.next()`. In the proxy the header comes from the same place: `request.headers.get("authorization")`.

Then **delete the check from `POST`**. One place decides who gets in. If the check lives in two places, one day they'll disagree.

The rule from lecture: **what the proxy returns decides what happens next.** `NextResponse.next()` means carry on to the route. A `Response.json(..., {status: 401})` means stop here, and the route never runs.

TEST IN POSTMAN

SEND	AUTH / BODY	EXPECT	BODY
`POST /api/pets`	No Auth, Nugget body	**401**	`{"error": "Missing or wrong token"}`
`POST /api/pets`	Bearer token, Nugget body	**201**	Nugget
`PUT /api/pets/3`	No Auth, full body	**401**	Locked now, and you didn't touch the PUT file
`PUT /api/pets/3`	Bearer token, full body	**200**	Pepper
`GET /api/pets`	No Auth	**401**	Wait. The public list is locked too.
`GET /api/health`	No Auth	**401**	So is the health check.

The proxy runs before **every** request, so it locked **everything**. The public can't even browse the animals. That's the next step.

Stuck? Code C7: the proxy as a gate

ASK YOUR CLAUDE

> *"In a Next 16 proxy, what's the difference between returning* `NextResponse.next()` *and returning* `Response.json({...}, {status: 401})` *? What happens to the route in each case?"*

8 Let the public routes back in

Give the proxy a list of the routes anyone may use, and check that list **first**. Public means **reading** only. `GET /api/pets` is public, but `POST /api/pets` is the same address with a different verb, and it must stay locked. So the check needs both the verb and the address.

At the top of `proxy.ts`, above the `proxy` function:

```
// Anyone may READ these without a token. Writing to them still needs one.
const publicPaths = ["/api/health", "/api/pets"];

function isPublic(method: string, path: string) {
  if (method !== "GET") return false;
  return publicPaths.includes(path) || path.startsWith("/api/pets/");
}
```

Why the extra `startsWith`? `/api/pets/2`, `/api/pets/3` and so on can't all be typed into a list, so everything under `/api/pets/` counts too.

Then, inside `proxy`, before the token check: if `isPublic(request.method, request.nextUrl.pathname)` is true, `return NextResponse.next()` straight away.

THE TEACHING POINT: DENY BY DEFAULT

You listed what's **open** and locked everything else. The other way round (list what's locked, leave everything else open) feels the same today. But the day someone adds `/api/pets/delete-all` and forgets to add it to the "locked" list, it's open to the world. With your way, a new route you forget about is locked until someone decides otherwise. When you forget, it stays safe.

TEST IN POSTMAN: RE-RUN YOUR WHOLE COLLECTION

SEND	AUTH / BODY	EXPECT	WHY
`GET /api/health`	No Auth	**200**	Public
`GET /api/pets`	No Auth	**200**	Public

`GET /api/pets/2`	No Auth	**200**	Public (starts with `/api/pets/`)
`GET /api/pets/99`	No Auth	**404**	Public, so it reached the route, which said "not found"
`POST /api/pets`	No Auth, Nugget body	**401**	Same address, but not a GET
`POST /api/pets`	Bearer token, Nugget body	**201**	Staff
`PUT /api/pets/3`	No Auth, full body	**401**	Not a GET
`GET /api/nope`	No Auth	**401**	Not on the list, so locked by default (not 404!)

Stuck? Code C8: the proxy with public routes

ASK YOUR CLAUDE

> *"Why is it safer to keep a list of public routes and lock everything else, instead of a list of private routes with everything else open? Give me a realistic example of the second way going wrong."*

> *"Here's my isPublic function: [paste it]. Without fixing anything, trace what it returns for GET /api/pets, POST /api/pets, GET /api/pets/2, GET /api/petsxyz and GET /api/admin/summary."*

9 Logging that helps

`proxy saw: GET /api/pets` proved the proxy was running. It isn't enough once the proxy is making decisions. When someone says "I got a 401 at about 2:15," you want to find that line and see **why**. Good log lines follow a few rules:

1. **One line per request, the same shape every time**, so your eye (or a search) can scan down them.
2. **When, what verb, what address** at the start of every line. `new Date().toISOString()` gives you a timestamp like `2026-10-08T18:59:52.845Z`.
3. **Say what was decided and why**: `-> public`, `-> allowed (token)`, `-> 401 missing or wrong token`.
4. `console.warn` **for things that went wrong** (a 401, a bad body), `console.log` for normal traffic. Later, real log tools can filter on that.
5. **Log when the data changes**: `created pet 7 (Nugget)`, `updated pet 3 (Pepper)`. Don't bother logging every read. The proxy already has those.
6. **Never log a secret.** Not the token, not a password, not the whole `Authorization` header.

That last rule is the one that gets people fired. Logs get pasted into chat, screenshotted for help, and shipped off to logging services that lots of people can read. A line like this hands your key to every one of them:

```
console.log("auth header:", request.headers.get("authorization"));   // NEVER
```

Log **what kind** of check happened and **what was decided**, never the value that was checked.

9a. In the proxy

Build the start of the line once, at the top of `proxy`, and use it on every path out of the function:

```
const path = request.nextUrl.pathname;
const line = `${new Date().toISOString()} ${request.method} ${path}`;

// ...then before each return:
console.log(line, "-> public");
console.warn(line, "-> 401 missing or wrong token");
console.log(line, "-> allowed (token)");
```

Delete the old `proxy saw:` line.

9b. In the routes

In `POST`, right after the push: `console.log(`created pet ${pet.id} (${pet.name})`)`. In `PUT`, after the changes: `updated pet ...`. Before each 400, a `console.warn` that says which check failed, like `console.warn("POST /api/pets: bad species", body.species)`. The species someone typed isn't a secret, so it's fine to log, and it's exactly what you'd want to know.

TEST: RUN YOUR COLLECTION, THEN READ THE TERMINAL

It should read like a story. Something like this (your times will differ):

```
2026-10-08T18:59:52.845Z POST /api/pets -> 401 missing or wrong token
2026-10-08T18:59:52.958Z POST /api/pets -> allowed (token)
created pet 7 (Nugget)
 POST /api/pets 201 in 27ms (next.js: 9ms, proxy.ts: 9ms, application-code: 9ms)
2026-10-08T18:59:53.275Z POST /api/pets -> allowed (token)
POST /api/pets: no name
 POST /api/pets 400 in 19ms (next.js: 6ms, proxy.ts: 7ms, application-code: 7ms)
2026-10-08T18:59:53.733Z GET /api/pets/7 -> public
 GET /api/pets/7 200 in 38ms (next.js: 23ms, proxy.ts: 8ms, application-code: 8ms)
2026-10-08T18:59:53.870Z PUT /api/pets/3 -> allowed (token)
updated pet 3 (Pepper)
 PUT /api/pets/3 200 in 22ms (next.js: 7ms, proxy.ts: 8ms, application-code: 7ms)
```

Look at the first line. The 401 has **no** Next line after it. A request the proxy turns away never reaches Next's own log, so your line is the only record it ever happened. That's why the proxy logs every decision, not only the successes.

Then search your terminal for `shelter-staff-2026`. It must not be there.

Stuck? Code C9: the proxy with good logging · C10: POST with log lines · C11: PUT with log lines

ASK YOUR CLAUDE

> *"Here are some lines from my server's terminal: [paste them]. If I were debugging a complaint like 'my request failed around 2:15,' what's good about these lines and what's missing? List things that should never appear in a log, and why."*

> *"What's the difference between console.log, console.warn and console.error, and when would a real logging service care which one I used?"*

10 Errors that help

Every error your API sends should tell the caller two things: **whose fault it was** (the status code) and **what to do about it** (the message). You already do that for the problems you thought of. This step is about the ones you didn't.

10a. Break it

Send a `POST /api/pets` with your Bearer token, Body set to raw JSON, and this as the whole body:

```
null
```

`null` is perfectly valid JSON, so `request.json()` doesn't throw, and your `try` / `catch` from Step 3 lets it through. Then `body.name` asks for a property of nothing, and the whole function crashes.

WHAT YOU SHOULD SEE

Postman: **500** with an **empty body**. Whoever called your API now knows only "something broke," not what or why. The terminal knows exactly (your line numbers may differ):

```
⨯ TypeError: Cannot read properties of null (reading 'name')
    at POST (app\api\pets\route.ts:19:19)
> 19 |   if (typeof body.name !== "string" || body.name === "") {
```

Try the same on `PUT /api/pets/2`. Same crash.

10b. Two kinds of error

	THE CLIENT'S FAULT	OUR FAULT
Examples	Bad JSON, a missing field, a wrong token, a pet that doesn't exist	A bug in our code, something we never expected
Status	4xx: 400, 401, 404	500
Message	Exactly what's wrong and how to fix it: `"age must be a number, 0 or more"`	Short and plain: `"Something went wrong on our side"`
Log	`console.warn`, saying which check failed	`console.error` with the **whole** error, so you can find the line

The `null` body is the client's mistake, but your API answered "500, our fault" and said nothing useful. Two fixes, in this order.

10c. Fix 1: a safety net

Wrap **everything** inside `POST` in one more `try` / `catch`. The inner one you already have still catches the failure you planned for (a body that isn't JSON). The new outer one catches every failure you didn't:

```
export async function POST(request: Request) {
  try {
    let body;
    try {
      body = await request.json();
    } catch {
      return Response.json({ error: "The body must be JSON" }, { status: 400 });
    }

    // ...all your checks, the push and the 201, unchanged...
  } catch (error) {
    console.error("POST /api/pets crashed:", error);
    return Response.json({ error: "Something went wrong on our side" }, { status: 500 });
  }
}
```

Send `null` again. Now Postman gets a **500** with `{"error": "Something went wrong on our side"}`, the same shape as every other error. The terminal has your line with the full details:

```
POST /api/pets crashed: TypeError: Cannot read properties of null (reading 'name')
    at POST (app\api\pets\route.ts:22:21)
```

THE DETAILS GO IN THE LOG, NEVER IN THE RESPONSE

It's tempting to send `error.message` back so the caller "knows what happened." Don't. Error messages and stack traces show your file names, your code, and sometimes data that's none of the caller's business. The caller gets a plain sentence. You get the details, in your own terminal. Same rule as Step 9: the log is for you.

10d. Fix 2: turn the 500 into a 400

Read that log line. It says `body` was `null`. That's not a bug in your code, it's bad input, and bad input is a 400 that tells the caller what to send. Add a check right after reading the body, before the `name` check:

```
// null is valid JSON, but body.name would crash on it.
if (body === null || typeof body !== "object") {
  return Response.json(
    { error: "The body must be a JSON object with name, species and age" },
    { status: 400 }
  );
}
```

That's the habit: **whenever the safety net catches something that was really the client's mistake, add a check so it never gets that far again.** The safety net stays for whatever you haven't thought of yet.

Now do both fixes in `PUT`. Its message should list all four fields: `"The body must be a JSON object with name, species, age and adopted"`.

The `GET` routes don't need any of this. Nothing a caller sends can make `find()` or `Response.json(pets)` throw. Put `try` / `catch` where things can really fail: reading a body, decoding something (Step 11), and next week, talking to a database.

TEST IN POSTMAN

SEND	AUTH / BODY	EXPECT	BODY
`POST /api/pets`	Bearer token, raw JSON `null`	**400**	`{"error": "The body must be a JSON object with name, species and age"}`
`POST /api/pets`	Bearer token, raw JSON `[]`	**400**	`{"error": "name is required"}`
`POST /api/pets`	Bearer token, Text `banana`	**400**	`{"error": "The body must be JSON"}`
`POST /api/pets`	Bearer token, Nugget body	**201**	Nugget. The happy path still works.

`PUT /api/pets/2`	Bearer token, raw JSON `null`	**400**	`{"error": "The body must be a JSON object with name, species, age and adopted"}`
`PUT /api/pets/99`	Bearer token, raw JSON `null`	**404**	`{"error": "No pet with id 99"}` . The pet check comes first.

Prove the safety net works. Add `throw new Error("testing the safety net");` as the first line inside the outer `try` of `POST` and send any POST. You should get the 500 JSON in Postman and `POST /api/pets crashed: Error: testing the safety net` in the terminal. Then **delete that line**.

Stuck? Code C12: POST with both fixes · C13: PUT with both fixes

ASK YOUR CLAUDE

> *"Here's my POST with a try/catch inside another try/catch: [paste it]. Without changing it, walk me through which catch runs, and what the client gets back, if the body is the word banana, if it's null, and if my own code has a typo that throws."*

> *"Why shouldn't an API send `error.message` or the stack trace back to the client in a 500? What could leak?"*

> *"For each of these, should my API answer 400, 401, 404 or 500, and why: a body of null, a missing token, a pet id that doesn't exist, a bug in my own code."*

11 The manager's door: Basic auth

The manager wants a summary, but a staff token shouldn't be enough to see it. Managers log in with a **username and password**. The oldest, simplest way to send those over HTTP is **Basic auth**, and you're going to check it by hand.

11a. The summary route

Make `app/api/admin/summary/route.ts` with a `GET` that works its answer out from the live list:

- `total` : how many pets
- `adopted` : how many have `adopted: true` . Use `pets.filter(...)` , then `.length` .
- `waiting` : total minus adopted

Restart the server first (`Ctrl` + `C` , `npm run dev`) so the list is fresh and your numbers match these. Then test it with your Bearer token: **200**, `{"total":6,"adopted":1,"waiting":5}` . Without a token: 401. Right now it's just another locked route.

11b. Add the manager's login to `.env.local`

```
API_TOKEN=shelter-staff-2026
ADMIN_USER=admin
ADMIN_PASS=adopt-dont-shop
```

11c. What Basic auth actually sends

The client glues the username and password together with a colon, then **base64-encodes** the result, and sends that after the word `Basic`:

```
admin:adopt-dont-shop                       glued together with a colon
YWRtaW46YWRvcHQtZG9udC1zaG9w                base64-encoded
Authorization: Basic YWRtaW46YWRvcHQtZG9udC1zaG9w
```

Try it yourself. Open any browser tab's DevTools console and type `btoa("admin:adopt-dont-shop")`, then `atob("YWRtaW46YWRvcHQtZG9udC1zaG9w")`. `btoa` encodes and `atob` decodes, and both exist in the browser **and** on your Next server.

> **BASE64 IS NOT ENCRYPTION**
>
> You just decoded a password with one function and no key. Base64 only turns any text into safe, plain letters for a header. Anyone who sees the header has the password. That's why Basic auth is only ever OK over **HTTPS**, where the whole request is encrypted on the way. `localhost` is fine for learning.

11d. Check it in the proxy

The server does the client's steps backwards. Add a new block to `proxy` for any address that starts with `/api/admin`. Put it **after** the public check and **before** the token check. (Think about why the order matters: what would the token check do to a request carrying `Basic ...`?)

1. **Accept the header**: `const auth = request.headers.get("authorization") ?? "";`. The `?? ""` turns "no header" into an empty string, so the next line can't crash.
2. If it doesn't start with `"Basic "`, answer 401.
3. **Base64-decode** the part after `"Basic "`: `atob(auth.slice("Basic ".length))` gives you `"admin:adopt-dont-shop"`. But `atob` **throws** if that text isn't real base64, so this is a Step 10 moment: wrap it in `try` / `catch`. Junk in the header is the client's mistake, so the `catch` answers **401** with a message that says what was expected, not a 500. (See the snippet below.)
4. **Split** on the colon: `const [user, pass] = decoded.split(":");`
5. **Compare** both to `process.env.ADMIN_USER` and `process.env.ADMIN_PASS`. Either one wrong: 401.

6. Both right: log it and `return NextResponse.next()`.

The decode, with its `try` / `catch`. `decoded` is declared outside the `try` so the lines after it can use it:

```
let decoded;
try {
  decoded = atob(auth.slice("Basic ".length));
} catch {
  return Response.json(
    { error: "Basic auth must be base64 of username:password" },
    { status: 401, headers: askForLogin }
  );
}

const [user, pass] = decoded.split(":");
```

Every admin 401 sends one extra header, which tells a browser to pop up its login box. Make it once at the top of the admin block and reuse it on every 401:

```
const askForLogin = { "WWW-Authenticate": 'Basic realm="admin"' };

return Response.json(
  { error: "Wrong username or password" },
  { status: 401, headers: askForLogin }
);
```

Log the decision like every other path (`-> 401 wrong admin login (user "admin")`, `-> allowed (admin "admin")`). The username is OK to log. **The password never is.**

TEST IN POSTMAN

The numbers below assume a fresh list, so restart the server once more before you start. To send a login: **Authorization** tab, **Auth Type** = **Basic Auth**, and fill in **Username** and **Password**. Then check the hidden **Headers**. Postman did the gluing and base64-ing for you, and you should see exactly `Basic YWRtaW46YWRvcHQtZG9udC1zaG9w`.

SEND	AUTH / BODY	EXPECT	BODY
`GET /api/admin/summary`	No Auth	**401**	`{"error": "Admin login required"}`
`GET /api/admin/summary`	Bearer `shelter-staff-2026`	**401**	`{"error": "Admin login required"}`. A staff token doesn't open the manager's door.
`GET /api/admin/summary`	Basic `admin` / `wrong`	**401**	`{"error": "Wrong username or password"}`

`GET /api/admin/summary`	No Auth, plus a hand-typed header on the Headers tab: `Authorization = Basic hello!`	**401**	`{"error": "Basic auth must be base64 of username:password"}`. Without the `try / catch` this is a 500.
`GET /api/admin/summary`	Basic `admin / adopt-dont-shop`	**200**	`{"total": 6, "adopted": 1, "waiting": 5}`
`POST /api/pets`	Bearer token, Nugget body	**201**	Nugget
`PUT /api/pets/3`	Bearer token, Pepper with `"adopted":true`	**200**	Pepper, adopted
`GET /api/admin/summary`	Basic, right login	**200**	`{"total": 7, "adopted": 2, "waiting": 5}`. The live model again.
`GET /api/pets`	No Auth	**200**	Public routes still public

Then try it in a **browser**: open http://localhost:3000/api/admin/summary. The `WWW-Authenticate` header makes the browser pop up its own login box. Type the username and password and you get the JSON. The browser then remembers them until you close it.

Stuck? Code C14: the summary route · C15: the finished proxy

ASK YOUR CLAUDE

> *"Explain HTTP Basic auth step by step: what exactly does the client send, and what does the server have to do to check it? What is base64, and why isn't it encryption?"*

> *"What does the WWW-Authenticate response header do, and why does my browser show a login box when it gets a 401 with* `WWW-Authenticate: Basic`*?"*

> *"In my proxy, I check /api/admin before I check the Bearer token. Walk me through what would happen to a request with* `Authorization: Basic ...` *if I swapped the order."*

What "done" looks like

- `GET /api/health`, `GET /api/pets` and `GET /api/pets/:id` work with no auth at all.
- `POST /api/pets` and `PUT /api/pets/:id` need `Authorization: Bearer shelter-staff-2026`, and check every field, answering 400 with `{"error": "..."}` when one is wrong.

- `GET /api/admin/summary` needs Basic auth with the username and password from `.env.local`, and a staff token is **not** enough.
- Every secret lives in `.env.local`, and `git status` doesn't list that file.
- There is **one** auth check, in `proxy.ts`. No route checks a token itself.
- The terminal shows one timestamped line per request with the decision, a line for every create and update, and never a token or password.
- Nothing you can send from Postman gets a 500 or an empty body. Bad input (`null`, `banana`, `Basic hello!`) is a 4xx with a message saying what to send instead. If your own code ever throws, POST and PUT answer `{"error": "Something went wrong on our side"}`, and the real error is in the terminal via `console.error`.
- `npm run build` finishes with no errors.

Going further

Finished early? Each of these is a real hole or a real feature. Try to explain what's going on **before** you fix it.

1. **The undefined token.** Comment out `API_TOKEN` in `.env.local` (put a `#` in front of it) and restart the server. In Postman, set Authorization to **No Auth** and add a header by hand on the Headers tab: `Authorization` = `Bearer undefined`. Send a POST. What happens, and why? (Hint: what does `` `Bearer ${process.env.API_TOKEN}` `` become when the variable is missing?) Make the proxy refuse everything if the token isn't set. Does the same hole exist for the admin login?
2. **A colon in the password.** Change `ADMIN_PASS` to `adopt:dont-shop` and try to log in with it. What does `"admin:adopt:dont-shop".split(":")` give you? Fix it so only the **first** colon splits. (Hint: `indexOf` and `slice`.)
3. **DELETE, and the id bug.** Add `DELETE` to `app/api/pets/[id]/route.ts` (staff only, answers 204 with no body; lecture's demo has one). Delete pet 2, then POST a new pet. What id does it get? Is that id already taken? Fix how ids are made.
4. **Filter the list.** Make `GET /api/pets?species=cat` return only cats. In a route, `new URL(request.url).searchParams.get("species")` reads it. Does the proxy still treat it as public? Why?
5. **A bigger summary.** Add `bySpecies` to the summary: `{"dog": 3, "cat": 2, "rabbit": 1}`.

Things that will bite you

- **Postman on the web can't reach `localhost`.** Use the desktop app.

- **Saving a file can reset the list.** While `npm run dev` runs, saving reloads the code, and the array goes back to what `types/pet.ts` says. That isn't your POST being broken. Re-run it.
- `proxy.ts` **in the wrong folder does nothing, silently.** Top folder, next to `package.json`.
- `.env.local` **in the wrong folder** means `process.env.API_TOKEN` is `undefined`, and every token is wrong. Same folder as `package.json`. The terminal lists `Environments: .env.local` at startup when it's found.
- **A 500 with an empty body means something threw.** Postman won't tell you what. The terminal will, down to the line. Read it before you change anything.
- **Body type set to Text instead of JSON** in Postman makes every POST a 400 "The body must be JSON".
- `"age": "2"` **is text.** `"age": 2` is a number. Same for `"adopted": "true"` vs `"adopted": true`.
- **The Authorization tab and a hand-typed Authorization header fight.** If you type the header yourself on the Headers tab, set the Authorization tab to **No Auth**.
- **A browser visit logs extra requests** like `GET /favicon.ico -> 401`. That's the browser asking for a tab icon, and your proxy correctly locking an address that isn't on the public list. Harmless.
- **Claude says** `middleware.ts`**.** That's the Next 15 name. In Next 16 it's `proxy.ts` with a function called `proxy`.
- **Port 3000 taken?** Next picks 3001 and says so in the terminal. Use whatever it says, or stop the other `npm run dev`.

Code reference

The full code at the end of each step. Try the step first; come here when you're stuck or want to compare. Each block says which file it goes in, and links back to its step.

C1 · The list route

↑ back to Step 1

app/api/pets/route.ts

```
import { pets } from "../../../types/pet";

export async function GET() {
  return Response.json(pets);
}
```

C2 · The one-pet route

↑ back to Step 2

app/api/pets/[id]/route.ts

```
import { pets } from "../../../../types/pet";

export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const pet = pets.find((p) => p.id === Number(id));
  if (!pet) {
    return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
  }
  return Response.json(pet);
}
```

C3 · The list route with POST

↑ back to Step 3

app/api/pets/route.ts

```
import { pets, allowedSpecies } from "../../../types/pet";
import type { Pet } from "../../../types/pet";

export async function GET() {
  return Response.json(pets);
}

export async function POST(request: Request) {
```

```
  let body;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  if (typeof body.name !== "string" || body.name === "") {
    return Response.json({ error: "name is required" }, { status: 400 });
  }
  if (!allowedSpecies.includes(body.species)) {
    return Response.json({ error: "species must be dog, cat or rabbit" }, { status: 400 });
  }
  if (typeof body.age !== "number" || body.age < 0) {
    return Response.json({ error: "age must be a number, 0 or more" }, { status: 400 });
  }

  const pet: Pet = {
    id: pets.length + 1,
    name: body.name,
    species: body.species,
    age: body.age,
    adopted: false,
  };
  pets.push(pet);
  return Response.json(pet, { status: 201 });
}
```

C4 · The one-pet route with PUT

↑ back to Step 4

app/api/pets/[id]/route.ts

```
import { pets, allowedSpecies } from "../../../../types/pet";

export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const pet = pets.find((p) => p.id === Number(id));
  if (!pet) {
    return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
  }
  return Response.json(pet);
}

// PUT replaces the WHOLE pet, so every field is required.
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
```

```
  const pet = pets.find((p) => p.id === Number(id));
  if (!pet) {
    return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
  }

  let body;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  if (typeof body.name !== "string" || body.name === "") {
    return Response.json({ error: "name is required" }, { status: 400 });
  }
  if (!allowedSpecies.includes(body.species)) {
    return Response.json({ error: "species must be dog, cat or rabbit" }, { status: 400 });
  }
  if (typeof body.age !== "number" || body.age < 0) {
    return Response.json({ error: "age must be a number, 0 or more" }, { status: 400 });
  }
  if (typeof body.adopted !== "boolean") {
    return Response.json({ error: "adopted must be true or false" }, { status: 400 });
  }

  pet.name = body.name;
  pet.species = body.species;
  pet.age = body.age;
  pet.adopted = body.adopted;
  return Response.json(pet);
}
```

C5 · The token check at the top of POST

↑ back to Step 5

`.env.local (next to package.json)`

```
API_TOKEN=shelter-staff-2026
```

`app/api/pets/route.ts (only the start of POST changes)`

```
export async function POST(request: Request) {
  // The token lives in .env.local and is only ever read here, on the server.
  if (request.headers.get("authorization") !== `Bearer ${process.env.API_TOKEN}`) {
    return Response.json({ error: "Missing or wrong token" }, { status: 401 });
  }

  let body;
  // ... the rest of POST stays exactly as it was in Step 3
}
```

C6 · The logging proxy

↑ back to Step 6

proxy.ts (top folder, next to package.json)

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  console.log("proxy saw:", request.method, request.nextUrl.pathname);
  return NextResponse.next();
}
```

C7 · The proxy as a gate

↑ back to Step 7

proxy.ts. Also delete the token check from POST in app/api/pets/route.ts (it's back to C3).

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  console.log("proxy saw:", request.method, request.nextUrl.pathname);

  // Return a response and the request stops here -- no route ever runs.
  if (request.headers.get("authorization") !== `Bearer ${process.env.API_TOKEN}`) {
    return Response.json({ error: "Missing or wrong token" }, { status: 401 });
  }

  // Return NextResponse.next() and it carries on to the route.
  return NextResponse.next();
}
```

C8 · The proxy with public routes

↑ back to Step 8

proxy.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

// Anyone may READ these without a token. Writing to them still needs one.
const publicPaths = ["/api/health", "/api/pets"];

function isPublic(method: string, path: string) {
  if (method !== "GET") return false;
  return publicPaths.includes(path) || path.startsWith("/api/pets/");
}

export function proxy(request: NextRequest) {
```

```
  const path = request.nextUrl.pathname;
  console.log("proxy saw:", request.method, path);

  if (isPublic(request.method, path)) {
    return NextResponse.next();
  }

  if (request.headers.get("authorization") !== `Bearer ${process.env.API_TOKEN}`) {
    return Response.json({ error: "Missing or wrong token" }, { status: 401 });
  }

  return NextResponse.next();
}
```

C9 · The proxy with good logging

↑ back to Step 9

proxy.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

// Anyone may READ these without a token. Writing to them still needs one.
const publicPaths = ["/api/health", "/api/pets"];

function isPublic(method: string, path: string) {
  if (method !== "GET") return false;
  return publicPaths.includes(path) || path.startsWith("/api/pets/");
}

export function proxy(request: NextRequest) {
  const path = request.nextUrl.pathname;
  // Same start to every line: when, what verb, what address. Never the token.
  const line = `${new Date().toISOString()} ${request.method} ${path}`;

  if (isPublic(request.method, path)) {
    console.log(line, "-> public");
    return NextResponse.next();
  }

  const auth = request.headers.get("authorization") ?? "";

  if (auth !== `Bearer ${process.env.API_TOKEN}`) {
    console.warn(line, "-> 401 missing or wrong token");
    return Response.json({ error: "Missing or wrong token" }, { status: 401 });
  }

  console.log(line, "-> allowed (token)");
  return NextResponse.next();
}
```

C10 · POST with log lines

↑ back to Step 9

app/api/pets/route.ts

```
import { pets, allowedSpecies } from "../../../types/pet";
import type { Pet } from "../../../types/pet";

// Public: anyone can see the list.
export async function GET() {
  return Response.json(pets);
}

// Private: needs the staff token (the proxy checks it before we get here).
export async function POST(request: Request) {
  let body;
  try {
    body = await request.json();
  } catch {
    console.warn("POST /api/pets: body was not JSON");
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  if (typeof body.name !== "string" || body.name === "") {
    console.warn("POST /api/pets: no name");
    return Response.json({ error: "name is required" }, { status: 400 });
  }
  if (!allowedSpecies.includes(body.species)) {
    console.warn("POST /api/pets: bad species", body.species);
    return Response.json({ error: "species must be dog, cat or rabbit" }, { status: 400 });
  }
  if (typeof body.age !== "number" || body.age < 0) {
    console.warn("POST /api/pets: bad age", body.age);
    return Response.json({ error: "age must be a number, 0 or more" }, { status: 400 });
  }

  const pet: Pet = {
    id: pets.length + 1,
    name: body.name,
    species: body.species,
    age: body.age,
    adopted: false,
  };
  pets.push(pet);
  console.log(`created pet ${pet.id} (${pet.name})`);
  return Response.json(pet, { status: 201 });
}
```

C11 · PUT with log lines

↑ back to Step 9

app/api/pets/[id]/route.ts

```
import { pets, allowedSpecies } from "../../../../types/pet";

// Public: anyone can look up one pet.
export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const pet = pets.find((p) => p.id === Number(id));
  if (!pet) {
    return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
  }
  return Response.json(pet);
}

// Private: PUT replaces the WHOLE pet, so every field is required.
// (PATCH would change only the fields you sent.)
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const pet = pets.find((p) => p.id === Number(id));
  if (!pet) {
    return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
  }

  let body;
  try {
    body = await request.json();
  } catch {
    console.warn(`PUT /api/pets/${id}: body was not JSON`);
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  if (typeof body.name !== "string" || body.name === "") {
    console.warn(`PUT /api/pets/${id}: no name`);
    return Response.json({ error: "name is required" }, { status: 400 });
  }
  if (!allowedSpecies.includes(body.species)) {
    console.warn(`PUT /api/pets/${id}: bad species`, body.species);
    return Response.json({ error: "species must be dog, cat or rabbit" }, { status: 400 });
  }
  if (typeof body.age !== "number" || body.age < 0) {
    console.warn(`PUT /api/pets/${id}: bad age`, body.age);
    return Response.json({ error: "age must be a number, 0 or more" }, { status: 400 });
  }
  if (typeof body.adopted !== "boolean") {
    console.warn(`PUT /api/pets/${id}: bad adopted`, body.adopted);
    return Response.json({ error: "adopted must be true or false" }, { status: 400 });
  }

  pet.name = body.name;
  pet.species = body.species;
```

```
  pet.age = body.age;
  pet.adopted = body.adopted;
  console.log(`updated pet ${pet.id} (${pet.name})`);
  return Response.json(pet);
}
```

C12 · POST with both fixes

↑ back to Step 10

app/api/pets/route.ts (finished)

```
import { pets, allowedSpecies } from "../../../types/pet";
import type { Pet } from "../../../types/pet";

// Public: anyone can see the list.
export async function GET() {
  return Response.json(pets);
}

// Private: needs the staff token (the proxy checks it before we get here).
export async function POST(request: Request) {
  // The outer try is the safety net: anything we didn't plan for becomes a clean 500.
  try {
    // The inner try catches the one failure we did plan for: a body that isn't JSON.
    let body;
    try {
      body = await request.json();
    } catch {
      console.warn("POST /api/pets: body was not JSON");
      return Response.json({ error: "The body must be JSON" }, { status: 400 });
    }

    // null is valid JSON, but body.name would crash on it.
    if (body === null || typeof body !== "object") {
      console.warn("POST /api/pets: body was not an object", body);
      return Response.json(
        { error: "The body must be a JSON object with name, species and age" },
        { status: 400 }
      );
    }
    if (typeof body.name !== "string" || body.name === "") {
      console.warn("POST /api/pets: no name");
      return Response.json({ error: "name is required" }, { status: 400 });
    }
    if (!allowedSpecies.includes(body.species)) {
      console.warn("POST /api/pets: bad species", body.species);
      return Response.json({ error: "species must be dog, cat or rabbit" }, { status: 400 });
    }
    if (typeof body.age !== "number" || body.age < 0) {
      console.warn("POST /api/pets: bad age", body.age);
      return Response.json({ error: "age must be a number, 0 or more" }, { status: 400 });
    }
```

```
    const pet: Pet = {
      id: pets.length + 1,
      name: body.name,
      species: body.species,
      age: body.age,
      adopted: false,
    };
    pets.push(pet);
    console.log(`created pet ${pet.id} (${pet.name})`);
    return Response.json(pet, { status: 201 });
  } catch (error) {
    // Our fault, not the client's. The details go in OUR log, never in the response.
    console.error("POST /api/pets crashed:", error);
    return Response.json({ error: "Something went wrong on our side" }, { status: 500 });
  }
}
```

C13 · PUT with both fixes

↑ back to Step 10

app/api/pets/[id]/route.ts (finished)

```
import { pets, allowedSpecies } from "../../../../types/pet";

// Public: anyone can look up one pet.
export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const pet = pets.find((p) => p.id === Number(id));
  if (!pet) {
    return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
  }
  return Response.json(pet);
}

// Private: PUT replaces the WHOLE pet, so every field is required.
// (PATCH would change only the fields you sent.)
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;

  // The outer try is the safety net: anything we didn't plan for becomes a clean 500.
  try {
    const pet = pets.find((p) => p.id === Number(id));
    if (!pet) {
      return Response.json({ error: `No pet with id ${id}` }, { status: 404 });
    }

    let body;
```

```
    try {
      body = await request.json();
    } catch {
      console.warn(`PUT /api/pets/${id}: body was not JSON`);
      return Response.json({ error: "The body must be JSON" }, { status: 400 });
    }

    if (body === null || typeof body !== "object") {
      console.warn(`PUT /api/pets/${id}: body was not an object`, body);
      return Response.json(
        { error: "The body must be a JSON object with name, species, age and adopted" },
        { status: 400 }
      );
    }
    if (typeof body.name !== "string" || body.name === "") {
      console.warn(`PUT /api/pets/${id}: no name`);
      return Response.json({ error: "name is required" }, { status: 400 });
    }
    if (!allowedSpecies.includes(body.species)) {
      console.warn(`PUT /api/pets/${id}: bad species`, body.species);
      return Response.json({ error: "species must be dog, cat or rabbit" }, { status: 400 });
    }
    if (typeof body.age !== "number" || body.age < 0) {
      console.warn(`PUT /api/pets/${id}: bad age`, body.age);
      return Response.json({ error: "age must be a number, 0 or more" }, { status: 400 });
    }
    if (typeof body.adopted !== "boolean") {
      console.warn(`PUT /api/pets/${id}: bad adopted`, body.adopted);
      return Response.json({ error: "adopted must be true or false" }, { status: 400 });
    }

    pet.name = body.name;
    pet.species = body.species;
    pet.age = body.age;
    pet.adopted = body.adopted;
    console.log(`updated pet ${pet.id} (${pet.name})`);
    return Response.json(pet);
  } catch (error) {
    // Our fault, not the client's. The details go in OUR log, never in the response.
    console.error(`PUT /api/pets/${id} crashed:`, error);
    return Response.json({ error: "Something went wrong on our side" }, { status: 500 });
  }
}
```

C14 · The summary route

↑ back to Step 11

app/api/admin/summary/route.ts

```
import { pets } from "../../../../types/pet";

// Admin only: the proxy wants a username and password (Basic auth)
// for anything under /api/admin.
```

```
// The numbers are worked out from the live list, so they change after every POST and PUT.
export async function GET() {
  const adopted = pets.filter((p) => p.adopted).length;
  return Response.json({
    total: pets.length,
    adopted: adopted,
    waiting: pets.length - adopted,
  });
}
```

C15 · The finished proxy

↑ back to Step 11

.env.local

```
API_TOKEN=shelter-staff-2026
ADMIN_USER=admin
ADMIN_PASS=adopt-dont-shop
```

proxy.ts (finished)

```
// Goes in the top folder, next to package.json. Anywhere else, Next never calls it.
// Every request passes through here before it reaches a route.
// Needs .env.local with API_TOKEN, ADMIN_USER and ADMIN_PASS.
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

// Anyone may READ these without a token. Writing to them still needs one.
const publicPaths = ["/api/health", "/api/pets"];

function isPublic(method: string, path: string) {
  if (method !== "GET") return false;
  return publicPaths.includes(path) || path.startsWith("/api/pets/");
}

export function proxy(request: NextRequest) {
  const path = request.nextUrl.pathname;
  // Same start to every line: when, what verb, what address. Never the token or password.
  const line = `${new Date().toISOString()} ${request.method} ${path}`;

  // 1. Public routes: let them straight through.
  if (isPublic(request.method, path)) {
    console.log(line, "-> public");
    return NextResponse.next();
  }

  const auth = request.headers.get("authorization") ?? "";

  // 2. Admin routes: a username and password (Basic auth).
  if (path.startsWith("/api/admin")) {
    // The WWW-Authenticate header makes a browser pop up its login box.
    const askForLogin = { "WWW-Authenticate": 'Basic realm="admin"' };
```

```
    if (!auth.startsWith("Basic ")) {
      console.warn(line, "-> 401 no admin login sent");
      return Response.json(
        { error: "Admin login required" },
        { status: 401, headers: askForLogin }
      );
    }

    // atob throws if the text isn't real base64.
    // That's the client's mistake, so it's a 401, not a 500.
    let decoded;
    try {
      decoded = atob(auth.slice("Basic ".length)); // "YWRtaW46..." -> "admin:password"
    } catch {
      console.warn(line, "-> 401 admin login was not base64");
      return Response.json(
        { error: "Basic auth must be base64 of username:password" },
        { status: 401, headers: askForLogin }
      );
    }

    const [user, pass] = decoded.split(":");

    if (user !== process.env.ADMIN_USER || pass !== process.env.ADMIN_PASS) {
      console.warn(line, `-> 401 wrong admin login (user "${user}")`);
      return Response.json(
        { error: "Wrong username or password" },
        { status: 401, headers: askForLogin }
      );
    }

    console.log(line, `-> allowed (admin "${user}")`);
    return NextResponse.next();
  }

  // 3. Everything else: the staff token (Bearer).
  if (auth !== `Bearer ${process.env.API_TOKEN}`) {
    console.warn(line, "-> 401 missing or wrong token");
    return Response.json({ error: "Missing or wrong token" }, { status: 401 });
  }

  console.log(line, "-> allowed (token)");
  return NextResponse.next();
}
```